

LeetCode 1493: Longest Subarray of 1's After Deleting One Element

 Yandex

 VK

 Sliding Window

✓ Valid Window

Condition: `zeroCount <= 1`

✗ Invalid Window

Condition: `zeroCount > 1`

The Core Concept & Difference from LC 1004

The logic here shares exactly the same idea as **LC 1004** (Max Consecutive Ones III), but with one key difference in how we calculate the final length.

LeetCode 1004

Answer was `r - l + 1`

Why? Because we were allowed to **flip** the zeros. The zero became a valid '1' and stayed in our window.

This Problem (LC 1493)

Hence, answer = `r - l`;

Why? One element is forcibly **deleted**. So one element literally disappears from the window length.

C++ Solution Code

```
class Solution {
public:
    int longestSubarray(vector<int>& nums) {
        int n = nums.size();
        int countone = 0;
        int ans = 0;
        int l = 0;
        int r = 0;
```

```

while(r < n) {
    if(nums[r] == 1) countone++;

    while((r - l + 1) - countone > 1) { // 1 se zyada 0 is invalid window
        if(nums[l] == 1) countone--;
        l++;
    }

    ans = max(ans, r - l + 1);
    r++;
}

return ans - 1; // because we are deleting one 0 entirely
}
};

```